Análisis y Reflexión Taller 3 IA

Integrantes del equipo:

- Carolina Ceballos Gómez - 202510267
- Juan Felipe Leon - 202510397
- Miguel Aguilar - 202411646
- Andrés Sanabria - 202411507

3b. Preguntas de análisis cualitativo

Responde las siguientes preguntas a partir del modelado de casos criminales en lógica de predicados y su experimentación con los motores de lógica proporcionados:

• ¿Cómo se modela la diferencia entre evidencia directa (por ejemplo, huellas en el arma) e indirecta (por ejemplo, un testimonio) usando predicados y reglas?

La diferencia se modela en la "distancia" de inferencia hasta llegar a culpable. La evidencia directa como huellas en el arma se conecta en un solo paso, de huellas_en(X, arma) y arma_del_crimen(arma) implica evidencia_directa(X). Por otra parte, la evidencia indirecta como un testimonio requiere pasos adicionales (reglas intermedias), primero hay que establecer que el testigo es confiable (descartado(T) implica testimonio_confiable(T)), y solo después de eso su acusación contribuye a la culpabilidad, lo que incrementa la profundidad del árbol de inferencia.

• ¿Qué sucede si un sospechoso tiene tanto evidencia en su contra como coartada verificada? ¿El motor detecta esta contradicción? Explica con un ejemplo.

El motor no detecta la contradicción como error, porque no puede derivar culpable(X). Por ejemplo, si en veneno_villa_espinas se agrega: “pythonkb.add_fact(Predicate("lejos_de_escena", (reynaldo,)))” Entonces descartado(reynaldo) se deriva, pero la regla de culpabilidad exige sin_coartada(reynaldo), que sigue siendo un hecho. El motor tiene ambos hechos en la kb simultáneamente sin conflicto, pero como ninguna regla lleva de descartado y evidencia_directa a culpable, simplemente falla al probar culpable(reynaldo). Así que no detecta la contradicción porque la manera en la que opera es simplemente acumular los hechos en la kb sin verificar su consistencia

• ¿En qué se diferencia razonar con forward chaining vs. backward chaining al resolver un caso? ¿Cuándo conviene usar cada uno?

El forward chaining parte de todos los hechos y aplica reglas iterativamente hasta saturar la KB. Es útil cuando se quiere un panorama completo del caso, por ejemplo, qué culpables hay, qué descartados, qué relaciones se derivan, etc. Su costo es que deriva todo, incluso lo irrelevante para la pregunta.

El backward chaining parte de una meta específica y busca hacia atrás solo lo necesario para probarla. Es más eficiente para responder preguntas concretas y a su vez produce un árbol de prueba legible que muestra exactamente por qué se llegó a esa conclusión.

En la práctica, conviene usar forward chaining para la exploración inicial ya que genera toda la información inicial y backward chaining, que genera subobjetivos para irlos probando y así verificar hipótesis específicas o entender una conclusión. Aunque para tomar la decisión de cuál elegir, podemos guiarnos por el tamaño de la kb, donde backward chaining es de mayor utilidad o por la meta planteada, pues si no hay una meta clara sería preferible usar forward chaining.

• ¿Qué rol juegan las variables vs. las constantes en la expresividad de las reglas? ¿Qué se perdería si solo se usaran constantes?

Las variables sirven para referirse a cualquier individuo o objeto dentro del dominio en general. Mientras que las constantes, en cambio, son específicas, en donde únicamente se refieren a una cosa per se. Ahora bien, si solo se utilizaran constantes, se perdería principalmente la generalización, teniendo como consecuencia escribir la misma regla una y otra vez para cada individuo. Tampoco se podrían usar las mismas reglas en distintos casos, sino que tocaría replantear todo cada vez.
Y por último, se dejaría de encontrar, al menos en este caso, el sospechoso por sí solo. Sino que se tendría que adoptar una forma parcialmente determinista, por el hecho de que se tendría que saber a quién mirar desde el inicio, por lo que dejaría de tener sentido usar un motor de inferencia.

• ¿Cómo afecta la suposición de mundo cerrado (lo que no se puede probar es falso) a las conclusiones sobre sospechosos que no tienen evidencia en su contra ni coartada verificada?
La suposición de mundo cerrado hace que todo lo que no esté explícitamente probado se tome como falso. Esto puede llevar a conclusiones problemáticas y en la práctica pueden ser engañosas. La falta de información no significa inocencia, solo significa que no se ha registrado. Por ejemplo, si no hay evidencia de que "tiene coartada", el sistema asume que no tiene coartada, o viceversa, y eso puede afectar las conclusiones. En un caso más delicado si todavía no hemos investigado a un sospechoso , no hay evidencia de que "es culpable" (aún) y el sistema no puede demostrar culpabilidad, es decir, la ausencia de información en mundos cerrados limita el modelo.

Punto 4: Reflexión
•Para cada uno de los puntos anteriores, explique brevemente: ¿Qué enfoque utilizó para su implementación? ¿Qué dificultades encontró? ¿Cómo verificó que su implementación era correcta?

1. **Model Checking:** me enfoqué en demostrar las condiciones que me pedían como con "casos", aunque en la primera función me pedían generar TODOS los modelos (para n átomos, debe generar 2 modelos), lo hice asimilándolo a un algoritmo de fuerza bruta, pues en clase nos explicaron que model checking era algo como eso, llenar todos los modelos y sus valores de verdad. En la segunda, me fui más por una salida temprana al encontrar el que cumplía. Mientras que en el tercero, que me pedían mirar si era tautología, en vez de revisar que cada uno sí lo fuera, me decidí enfocar en encontrar un contraejemplo y devolver False apenas encontrara uno. Para check_entailment, hice algo similar y busqué un contraejemplo donde toda la KB sea verdadera y la query sea falsa. Las dificultades fueron especialmente en check_entailment, pues necesitaba primero verificar que estuviera revisando todos los atoms de la kb y de la query, este fue con el que más me demoré e incluso pasé al último antes de finalizarlo, pero lo solucioné usando sets y logré implementar el resto de la función. La verifiqué guiándome por las pruebas (que sí pasara todos los tests) pero además lo pensé como en lenguaje natural, iba revisando cada línea que era lo que estaba haciendo y si sí seguía model checking.

2. **Conversión a CNF:** El enfoque que utilicé para resolver las expresiones a CNF fue identificar cómo interactúa cada operador lógico según la regla de inferencia (Doble negación, Leyes de Morgan). La mayor dificultad hallada fue entender cómo funcionaban las clases, qué atributos y métodos tenían. Igualmente, fue difícil diseñar cada función de manera recursiva. Para verificar que las pruebas estuvieran correctas, se aplicaron las pruebas diseñadas desde un inicio que corroboraban que las fórmulas estuvieran correctas y no se modificaría el sentido de las oraciones.

3. **Lógica de Predicados:** Como tal el enfoque que seguir para implementar de manera correcta, fue primero, para arrancar a modelar el problema, empecé por identificar las partes esenciales del primer párrafo y utilizando la función de add_fact() con el fin de tener un registro de los hechos, luego con add_rule() plantear las precondiciones y la conclusión, con el objetivo de modelar el problema y plantear todo el conocimiento base. Ahora, las principales dificultades que tuve, fueron para red_puerto_sombras realizar de manera adecuada los predicados, más que todo por lo que tocaba encadenar varios y si uno llegaba a fallar en la formulación del orden, ya quedaba mal toda la cadena. Ya para la verificación, primero seguir la estrategia que había visto en mayor profundidad en cursos anteriores (FMC), siguiendo la premisa de dividir y conquistar, y haciendo este proceso de forma manual para luego pasarla a código. Asimismo, también me iba guiando por los tests para saber si estaban quedando bien.

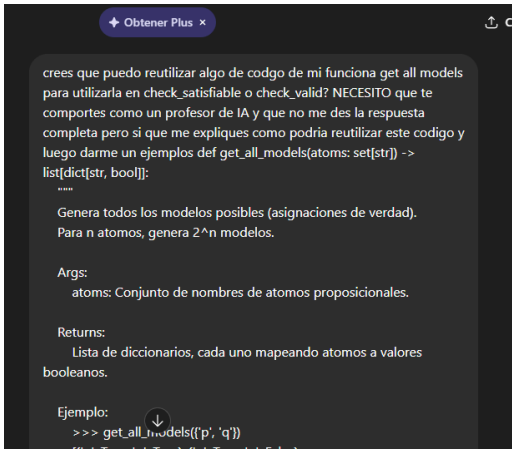
4. **Bono:** Diseñé la narrativa primero, es decir, quién es culpable y por qué, y luego trabajé hacia atrás. Definí qué predicados intermedios necesitaba para llegar al culpable mediante encadenamiento, asegurando que cada regla tuviera sus hechos base correspondientes. La

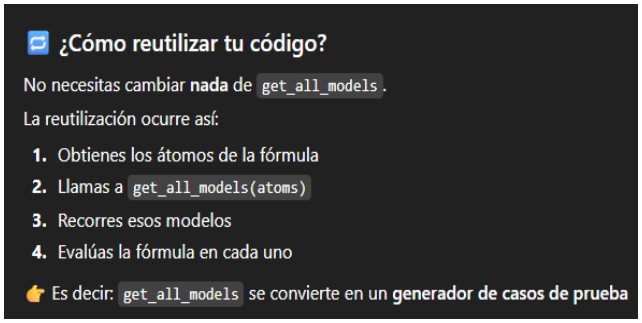
dificultad principal que encontré fue el balance entre originalidad y corrección lógica ya que no tendría mucho sentido el ejercicio si tocara repetir lo mismo que los casos anteriores y también claramente diseñar los ForallGoal y los demás elementos tal que fueran trazables y tuvieran sentido lógico. Para verificar, tracé manualmente el encadenamiento hacia atrás de cada consulta: para cada QuerySpec, verifiqué que existía una cadena de hechos y reglas que la satisface.

• Escriba una reflexión concisa sobre la co-construcción de la solución con apoyo de la IA, indicando qué aprendizajes obtuvo, si las correcciones de la IA fueron básicas o mejoras secundarias, y los beneficios y limitaciones de la IA en este contexto.

Este taller no se nos dificulta tanto como equipo, pues teníamos la mayoría de los conceptos claros (aunque algunos específicos sí le preguntamos a la IA) , además de que con las pruebas era mucho más fácil verificar si lo que estábamos implementando estaba funcionando correctamente. Así que en comparación con los talleres anteriores, no hicimos tanto uso de la IA. Las únicas preguntas que le hicimos era si había manera de hacer reutilización de código pues había algunas funciones muy similares en las que cambiaban solo algunos detalles, aunque las IA nos respondía cosas valiosas y nos ayudo a expandir nuestro conocimiento de programación en python respecto a la reutilización de código, no implementamos los cambios o mejoras que nos arrojaba la respuesta pues nos parecía que nuestras implementaciones estaban bien y tal vez si las cambiamos para no rehusar, se pondrían un poco confusas de entender. Creo que el beneficio de la IA es que al tener acceso a tanta información nos arroja datos muy útiles, pero siento que por su necesidad de responder el prompt enviado, no analizaba los cambios que sugería y que tal vez estos resultan aún más complejos o confusos en comparación con nuestra implementación actual. Adicionalmente, para el punto 3 fue de utilidad para hacer correcciones a errores sutiles que había hecho durante la planeación del problema. Principalmente la utilicé para entender errores de lógica, por lo que fue bastante útil acceder a la ayuda de la IA (Claude Opus 4.6) para comprender y realizar las correcciones pertinentes.

Esta fue la conversación con Chat-GPT4:





y entre def check_satisfiable(formula: Formula) -> tuple[bool, dict[str, bool] | None]:

```
"""
Determina si una formula es satisfacible.

Args:
    formula: Formula logica a verificar.

Returns:
    (True, modelo) si encuentra un modelo que la satisface.
```

🧠 Relación profunda entre ambas funciones

No son dos problemas distintos. En realidad:

> `check_valid` **es un caso especial de** `check_satisfiable`

Más formalmente:

$$\text{Una fórmula es válida} \iff \neg\text{fórmula es insatisfacible}$$

🔁 Dos formas de reutilizar código

Te voy a mostrar las dos (como lo haría un profe), pero tú decides cuál implementar.

✅ Opción 1: Misma estructura, distinta condición

Usas `get_all_models` en ambas.

- `check_satisfiable` → buscas *un modelo que funcione*
  👉 si encuentras uno donde la fórmula es `True`, paras.
- `check_valid` → buscas *un modelo que falle*
  👉 si encuentras uno donde la fórmula es `False`, paras.

💡 Diferencia clave:

- satisfiable = **∃ uno verdadero**
- valid = **∀ todos verdaderos**

🧠 Opción 2: Reutilización elegante

Defines `check_valid` usando `check_satisfiable`:

👉 Una fórmula es válida si su negación es insatisfacible

‹/› Python

```python
check_valid(formula) = not check_satisfiable(Not(formula))[0]
```

Conversacion con Claude Opus 4.6:

Introduce contrabando.
Quien comete fraude documental sin coartada es culpable.
Quien introduce contrabando sin coartada es culpable.
Dos personas comparten red si pertenecen al mismo cartel.
Si dos culpables comparten red, su actividad constituye una operación conjunta.
El testimonio de una persona descartada contra alguien es confiable.
Una red está activa si al menos uno de sus miembros es culpable.

Estoy teniendo un error al plantear la siguiente parte:
kb.add_rule(Rule(head=Predicate("operacion_conjunta", (Term("$X"), Term("$Y"))), body=(Predicate("culpable", (Term("$X"),)), Predicate("culpable", (Term("$Y"),)), Predicate("comparte_red", (Term("$X"), Term("$Y"))),)))

Show less

El error que describes es exactamente el que mencionamos antes: la regla permite que `$X` y `$Y` sean la misma persona. Pero hay otro error posible dependiendo de qué query estás intentando probar.

La query es:

```python
Predicate("operacion_conjunta", (Term("oficial_duarte"), Term("marinero_pinto")))
```

Para que esto funcione, `comparte_red(oficial_duarte, marinero_pinto)` debe poder derivarse. Verifica que tengas esta regla **antes** de `operacion_conjunta`:

```python
kb.add_rule(Rule(
    head=Predicate("comparte_red", (Term("$X"), Term("$Y"))),
    body=(
        Predicate("pertenece_cartel", (Term("$X"), Term("$C"))),
        Predicate("pertenece_cartel", (Term("$Y"), Term("$C"))),
```

Esta fue la conversación con Copilot de Visual Studio Code:

explicame, no hagas, la logica para desarrollar eliminate_iff

Optimizing tool selection...

Reviewed 3 files and searched for regex patterns

Explicación de `eliminate_iff()`

La idea es **eliminar todos los bicondicionales** (Iff) de una fórmula lógica, convirtiéndolos en su equivalencia más primitiva.

La transformación clave

Un bicondicional `Iff(a, b)` significa "a si y solo si b", que es equivalente a:

$$\text{Iff}(a, b) \equiv \text{And}(\text{Implies}(a, b), \text{Implies}(b, a))$$

Es decir: **(a → b) ∧ (b → a)**

El patrón recursivo

Debes recorrer toda la estructura de la fórmula usando **pattern matching** sobre el tipo:

1. **Si es** `Atom` : Devolvelo tal cual (caso base)
2. **Si es** `Not` : Aplica `eliminate_iff` recursivamente al operando y envuélvelo en `Not` nuevamente
3. **Si es** `And` : Aplica `eliminate_iff` recursivamente a **cada conjunct**, y construye un nuevo `And` con esos resultados
4. **Si es** `Or` : Aplica `eliminate_iff` recursivamente a **cada disjunct**, y construye un nuevo `Or` con esos resultados
5. **Si es** `Implies` : Aplica `eliminate_iff` recursivamente al antecedente y consecuente, y construye un nuevo `Implies` con esos resultados
6. **Si es** `Iff` (el caso especial):
   - Aplica `eliminate_iff` a `.left` y `.right` para obtener versiones sin bicondicionales
   - Construye dos `Implies` :
     - `Implies(left_transformada, right_transformada)`
     - `Implies(right_transformada, left_transformada)`
   - Envuelve ambas en un `And`